

Bubbles CLI-First Buildathon MVP Development Plan

For agentic workers: REQUIRED SUB-SKILL: Use `superpowers:subagent-driven-development` (recommended) or `superpowers:executing-plans` to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Build the buildathon MVP of Bubbles: a macOS desktop pet interface that lets users speak or type tasks, routes real work to MiniMax/CLI/MCP integrations, manages memory and permissions, and responds through one animated Bubbles avatar.

Architecture: Bubbles is the desktop UX, emotion, memory, onboarding, and permission layer. The CLI agent and MiniMax integrations are the worker brain for research, coding, email, calendar, creative, and tool-based tasks. MCP/connectors are exposed through a hybrid model where Bubbles owns connector UX and approvals while the CLI uses approved tools.

Tech Stack: Electron, React, TypeScript, Vite, PixiJS, SQLite, MiniMax API, MiniMax CLI (`mmx-cli`), MCP servers/connectors, macOS Keychain, Node child processes, Vitest, Playwright.

Current Implementation Status

- The integrated MVP mock test has passed all 14 live phases. See `docs/MVP-Mock-Test-Status.md` for phase evidence, fix log, and verification commands.
- Phase 1 is implemented and verified as the current Electron desktop shell: a draggable floating Bubbles avatar window, separate assistant workspace window, shared app state in Electron main, PixiJS avatar rendering, chat/workspace surfaces, settings/setup, and renderer coverage for avatar/panel interactions.
- Phase 2 is implemented and verified as a required dual-key MiniMax setup: users must provide a MiniMax General API key for direct API verification and a MiniMax Token Plan Key for `mmx-cli`; setup cannot complete in API-only mode.
- Phase 3 is implemented and verified as the CLI Agent Bridge: chat submits structured task packets through Electron main, runs the app-local MiniMax CLI, streams normalized task events into the task drawer, maps task lifecycle to avatar states, handles cancellation/timeouts/errors, and stores redacted task logs.
- Agent switching, Agent Birth, approval gating, fixture connector readiness, memory/timeline persistence, creative degraded messaging, cancellation, restart persistence, and redaction have all passed live mock testing.
- Post-test fixes are committed in the latest mock-test commit: early CLI cancellation handling, durable memory/timeline redaction, explicit memory echo redaction, and MiniMax/Agent Birth JSON normalization.
- Current verification commands:
 - `npm test`
 - `npm run typecheck`

- `npm run test:core -- cliBridge`
 - `npm run test:core -- memoryStore timelineStore redactSecrets`
 - `npm run test:desktop`
 - Next implementation work is hardening and expansion: real connector providers, voice/media artifact generation, packaging/signing, and broader automated coverage.
-

Product Constraints Locked For MVP

- Target platform is macOS first.
- Desktop stack is Electron + React + TypeScript + PixiJS.
- Integrations are real-first: MiniMax API, MiniMax CLI, and MCP connectors should be implemented against real interfaces.
- Test fixtures and connector simulators are allowed for automated tests and fallback demos, but the product architecture must support real integrations first.
- Avatar scope is one universal Bubbles sprite sheet only.
- Default desktop experience is a draggable floating Bubbles avatar, not a permanent dashboard panel.
- Clicking Bubbles opens a larger assistant panel in a second Electron window that should feel like a desktop Codex/Claude interface with conversations, active chat, history, task status, settings, memory, connectors, and approvals.
- Speech bubbles, task drawers, approvals, settings, setup, and memory are summoned surfaces around or inside the assistant panel; they are not always visible by default.
- Avatar window and assistant panel window share one Electron-main state for avatar state, messages, speech bubble text, and panel open/closed status.
- Agents do not change avatar color, costume, props, sprite sheet, or chat panel theme.
- Agents change badge/name, `skills.md`, voice style, response behavior, memory rules, safety rules, and allowed tools.
- The requested plan file path is `docs/dev_plan_mvp.md`.

MVP File Structure

```
bubbles-MVP/  
package.json  
pnpm-workspace.yaml  
tsconfig.base.json  
apps/  
  desktop/  
    package.json  
    index.html  
    vite.config.ts  
    electron.vite.config.ts
```

```
src/  
  main/  
    main.ts  
    preload.ts  
    ipc/  
      services/  
renderer/  
  App.tsx  
  components/  
    AssistantPanel.tsx  
    ConversationHistory.tsx  
    FloatingAvatarWindow.tsx  
    TaskDrawer.tsx  
  screens/  
  styles/  
  avatar/  
    AvatarStage.tsx  
    animationCatalog.ts  
    bubbles_mvp.json  
    bubbles_mvp.png  
packages/  
  core/  
    package.json  
    src/  
      agents/  
      approvals/  
      cli/  
      connectors/  
      emotion/  
      memory/  
      minimax/  
      orchestration/  
      security/  
      shared/  
      timeline/  
agents/  
  general-assistant/  
    agent.json  
    skills.md  
  research-agent/  
    agent.json  
    skills.md  
  coding-agent/  
    agent.json  
    skills.md  
  email-calendar-assistant/  
    agent.json  
    skills.md  
  creative-minimax-helper/  
    agent.json  
    skills.md  
data/  
  .gitkeep  
docs/  
  full_plan.md  
  full_plan_v2.md  
  dev_plan_mvp.md
```

Shared Data Contracts

Use these shared TypeScript contracts across phases. Keep them in `packages/core/src/shared/types.ts`.

```

export type AvatarState =
  | 'idle'
  | 'listening'
  | 'thinking'
  | 'working'
  | 'waiting_approval'
  | 'confused'
  | 'concerned'
  | 'celebrating'
  | 'sleeping';

export type TaskType =
  | 'general.plan'
  | 'research.web'
  | 'coding.project'
  | 'email.read'
  | 'email.reply'
  | 'calendar.read'
  | 'calendar.update'
  | 'agent.create'
  | 'creative.minimax';

export type ApprovalRisk = 'low' | 'medium' | 'high';

export interface AgentProfile {
  id: string;
  name: string;
  role: string;
  badgeName: string;
  voiceStyle: string;
  allowedTools: string[];
  memoryRules: string[];
  safetyRules: string[];
  responseStyle: string;
  skillsPath: string;
  createdAt: string;
  updatedAt: string;
}

export interface TaskPacket {
  taskId: string;
  userText: string;
  activeAgentId: string;
  taskType: TaskType;
  mode: 'plan_only' | 'plan_then_act' | 'act_after_approval';
  memoryContext: MemoryItem[];
  skillsMarkdown: string;
  allowedTools: string[];
  approvalPolicy: 'none' | 'preview_sensitive_actions' | 'preview_all_actions';
  outputPreference: {
    userLevel: 'nontechnical';
    responseStyle: 'clear_spoken_summary' | 'structured_report' |
'concise_status';
    includeTechnicalDetails: boolean;
  };
}

export interface CliEvent {
  taskId: string;
  type:
    | 'task.received'
    | 'task.status'
    | 'task.partial_output'
    | 'tool.requested'

```

```

    | 'approval.required'
    | 'approval.accepted'
    | 'approval.denied'
    | 'task.result'
    | 'task.error'
    | 'task.cancelled';
  payload: Record<string, unknown>;
  createdAt: string;
}

export interface ApprovalRequest {
  id: string;
  taskId: string;
  agentId: string;
  actionType:
    | 'send_email'
    | 'calendar_update'
    | 'file_write'
    | 'shell_command'
    | 'cli_install'
    | 'agent_file_create'
    | 'external_data_send';
  risk: ApprovalRisk;
  title: string;
  explanation: string;
  preview: Record<string, unknown>;
  status: 'pending' | 'approved' | 'denied';
  createdAt: string;
  resolvedAt?: string;
}

export interface MemoryItem {
  id: string;
  type:
    | 'user_preference'
    | 'project_context'
    | 'task_summary'
    | 'decision'
    | 'agent_history'
    | 'relationship'
    | 'connector_context'
    | 'approval_history';
  content: string;
  sourceTaskId?: string;
  agentId?: string;
  tags: string[];
  importance: 1 | 2 | 3 | 4 | 5;
  createdAt: string;
  updatedAt: string;
}

```

Phase 1: Floating Avatar Shell + One-Sheet Avatar

Goal: Create the desktop app foundation where Bubbles defaults to a draggable floating avatar in its own compact Electron window. Clicking Bubbles opens or closes a separate Codex/Claude-style assistant workspace window with chat, conversation history, settings/status, and task drawer placeholders.

Main files:

- Create `package.json`, `pnpm-workspace.yaml`, `tsconfig.base.json`.

- Create `apps/desktop/package.json`.
- Create `apps/desktop/src/main/main.ts`.
- Create `apps/desktop/src/main/preload.ts`.
- Create `apps/desktop/src/renderer/App.tsx`.
- Create `apps/desktop/src/renderer/App.test.tsx`.
- Create `apps/desktop/src/renderer/components/FloatingAvatarWindow.tsx`.
- Create `apps/desktop/src/renderer/components/AssistantPanel.tsx`.
- Create `apps/desktop/src/renderer/components/ConversationHistory.tsx`.
- Create `apps/desktop/src/renderer/components/TaskDrawer.tsx`.
- Create `apps/desktop/src/avatar/AvatarStage.tsx`.
- Create `apps/desktop/src/avatar/bubbles_mvp.json`.

Implementation tasks:

- ☐ Initialize Electron + Vite + React + TypeScript workspace.
- ☐ Add scripts: `dev`, `build`, `test`, `lint`, `typecheck`.
- ☐ Create a compact transparent always-on-top avatar Electron window with frameless mode and no visible window shadow/outline.
- ☐ Create a separate assistant workspace Electron window for the full Codex/Claude-style panel.
- ☐ Keep the avatar window above the assistant workspace window so Bubbles remains visible in front of the panel.
- ☐ Do not resize the avatar window into the panel; this creates unwanted transparent bridge/click layers between Bubbles and the workspace.
- ☐ Keep shared Phase 1 app state in Electron main: avatar state, messages, speech bubble text, and panel open/closed status.
- ☐ Make the default window content avatar-first: Bubbles should appear as a floating pet, not inside a full dashboard panel.
- ☐ Implement drag behavior so the user can move Bubbles anywhere on the desktop.
- ☐ Expose safe IPC bridge through preload. Do not expose raw Node APIs to renderer.
- ☐ Build a compact floating avatar surface with Bubbles, speech bubble, and pet name label **Bubbles**; do not show avatar mood text under the pet.
- ☐ Style the speech bubble as a clean solid bubble with subtle highlight/shadow; do not add transparent glass overlays around the pet.

- ☐ Clicking Bubbles toggles the assistant workspace window open/closed.
- ☐ Sending a local chat message in the assistant window updates the floating speech bubble through shared Electron state.
- ☐ Changing avatar state in the assistant window updates the visible floating avatar through shared Electron state.
- ☐ Build the assistant panel to resemble a desktop Codex/Claude interface:
 - conversation history/sidebar placeholder.
 - active chat transcript.
 - local-only message input.
 - task drawer/status section.
 - settings/status section.
 - memory/timeline placeholder.
 - connector status placeholder.
- ☐ Keep the panel closed by default after launch unless first-run setup needs attention.
- ☐ Add PIXIJS renderer for `bubbles_mvp.png` and `bubbles_mvp.json`.
- ☐ Define one sprite sheet layout:
 - 128x128 frames.
 - 1024x1152 sheet.
 - 8 columns, 9 rows.
 - `idle`: row 0, 6 frames.
 - `listening`: row 1, 6 frames.
 - `thinking`: row 2, 8 frames.
 - `working`: row 3, 8 frames.
 - `waiting_approval`: row 4, 6 frames.
 - `confused`: row 5, 6 frames.
 - `concerned`: row 6, 6 frames.
 - `celebrating`: row 7, 8 frames.
 - `sleeping`: row 8, 6 frames.
- ☐ Add local avatar state controls in development mode for all 9 states.
- ☐ Keep dev-only state controls hidden behind the assistant panel or a developer toggle so the default experience stays pet-like.
- ☐ Use `Bubbles` as the compact pet label in Phase 1. Later agent identity may change an in-panel active agent badge without changing sprite art or chat panel theme.

- ☐ Treat `celebrating` as a looping state that continues until another avatar state is selected.
- ☐ Add drag-and-drop hover styling for future file/folder/text drops; Phase 1 must not process files yet.

Acceptance criteria:

- ☐ `pnpm install` completes.
- ☐ `pnpm dev` opens a transparent macOS desktop window with a floating Bubbles avatar.
- ☐ User can drag the avatar/window to a new desktop position.
- ☐ Clicking Bubbles opens the assistant panel in a separate Electron window.
- ☐ Clicking Bubbles again closes the assistant panel.
- ☐ Closing the assistant panel returns to the compact floating avatar.
- ☐ Assistant panel can be dragged by its header and stays behind the floating avatar.
- ☐ No transparent bridge, outline, or connected layer appears between Bubbles and the assistant panel.
- ☐ Compact label under the pet says **Bubbles** only.
- ☐ Bubbles displays using one sprite sheet.
- ☐ Avatar can switch among all 9 states.
- ☐ Avatar state changes from the assistant panel update the visible floating avatar.
- ☐ `celebrating` continues looping until another avatar state is selected.
- ☐ The assistant panel shows conversation history placeholder, active chat, settings/status, task drawer, memory placeholder, and connector placeholder.
- ☐ Local chat input echoes user messages in the active chat and floating speech bubble.
- ☐ Compact mode shows only the **Bubbles** pet label; later agent/task status belongs in the assistant panel or future summoned surfaces.
- ☐ No agent-specific color, costume, prop, or chat theme code is introduced.

Tests:

- ☐ Unit test animation metadata parsing.
- ☐ Unit test invalid avatar state falls back to `idle`.
- ☐ Renderer test confirms the compact floating avatar renders by default.
- ☐ Renderer test confirms clicking Bubbles opens the assistant panel.
- ☐ Renderer test confirms clicking Bubbles again closes the assistant panel.
- ☐ Renderer test confirms conversation history, active chat, settings/status, memory placeholder, connector placeholder, speech bubble, pet label, and task drawer render.

- ☐ `Renderer` test confirms chat message state updates the floating speech bubble.
- ☐ `Renderer` test confirms assistant-panel mood controls update the visible avatar.
- ☐ `Renderer` test confirms `celebrating` remains active until another state is selected.
- ☐ `Renderer` test confirms the assistant panel drag handle requests window movement.
- ☐ `Renderer` test confirms closing the panel returns to compact mode.
- ☐ Manual test transparent window, dragging, click-open panel, and always-on-top behavior on macOS.

File responsibility notes:

- `App.tsx` is the React runtime component. It decides whether the current `renderer` window is the compact avatar window or the assistant workspace window, binds UI events, and syncs with Electron shared state.
- `App.test.tsx` is the Vitest/Testing Library test file for `App.tsx`. It is not shipped as app UI; it verifies expected Phase 1 behavior such as open/close, chat echo, panel drag, mood updates, and persistent `celebrating` playback.

Phase 2: First-Run MiniMax Setup

Goal: Let a normal user configure MiniMax safely with both required credentials: a MiniMax General API key for direct API calls and a MiniMax Token Plan Key for `mmx-cli`, then install/authenticate the CLI and verify real connectivity.

Status: Completed for the current app. The remaining Phase 2 item is manual testing with real MiniMax credentials on macOS.

Main files:

- Implemented `packages/core/src/minimax/minimaxApiClient.ts`.
- Implemented `packages/core/src/minimax/minimaxCliManager.ts`.
- Implemented `packages/core/src/minimax/setupService.ts`.
- Implemented `packages/core/src/security/secureKeyStore.ts`.
- Implemented `apps/desktop/src/renderer/screens/SetupScreen.tsx`.
- Implemented `apps/desktop/src/main/ipc/setupIpc.ts`.

Implementation tasks:

- ☒ Add setup state machine: `needs_general_api_key`, `verifying_general_api`, `needs_token_plan_key`, `checking_cli`, `needs_cli_install`, `authenticating_cli`, `verifying_cli`, `ready`, `setup_error`.
- ☒ Store MiniMax General API key and MiniMax Token Plan Key as separate macOS Keychain entries.
- ☒ Redact both keys in all logs and UI error messages.

- ☒ Implement General API key verification with a safe tiny direct request to `POST /v1/chat/completions`.
- ☒ Require the Token Plan Key before any completed setup state; do not offer single-key or API-only completion.
- ☒ Detect CLI availability using app-local `mmx`; Electron setup requires the app-local binary so global CLI auth/config cannot leak into the app.
- ☒ If `mmx` is missing, show approval UI before any install command.
- ☒ Implement user-approved app-local CLI install flow with a visible command preview.
- ☒ Authenticate MiniMax CLI with the Token Plan Key only; never pass the General API key to `mmx auth login`.
- ☒ Verify CLI with `mmx auth status --output json`, `mmx quota`, and `mmx text chat --message "Reply with exactly: ok"`.
- ☒ If CLI setup fails, keep setup incomplete and show a friendly redacted error.
- ☒ Save setup status locally, excluding secrets.

Acceptance criteria:

- ☒ User can enter and verify a MiniMax General API key.
- ☒ User can enter and verify a MiniMax Token Plan Key for CLI.
- ☒ Both keys are stored outside repo files and SQLite.
- ☒ Bubbles verifies direct API access with only the General API key.
- ☒ Bubbles authenticates MiniMax CLI with only the Token Plan Key.
- ☒ Bubbles detects whether MiniMax CLI exists.
- ☒ Bubbles never installs CLI without approval.
- ☒ Bubbles cannot reach `ready` unless both keys and CLI verification pass.
- ☒ Setup errors are friendly and do not expose secrets.

Tests:

- ☒ Unit test setup cannot reach `ready` with only one key.
- ☒ Unit test direct API verification never receives the Token Plan Key.
- ☒ Unit test CLI authentication never receives the General API key.
- ☒ Unit test both Keychain entries are stored and reset independently.
- ☒ Unit test secret redaction.
- ☒ Unit test CLI missing detection.
- ☒ Renderer test the two required key entry steps, CLI install approval, invalid Token Plan errors, and final ready state.

- ☒ Integration test API verification with a mocked HTTP client.
- ☐ Manual test with real MiniMax General API key, real MiniMax Token Plan Key, fresh app-local CLI install, `mmx quota`, and CLI text verification.

Manual Phase 2 checklist:

- ☐ Launch with `corepack pnpm dev`.
- ☐ Confirm setup asks first for **MiniMax General API key**.
- ☐ Confirm invalid General API key stays incomplete with a redacted error.
- ☐ Enter valid General API key and confirm setup advances to **MiniMax Token Plan Key for CLI**.
- ☐ Confirm chat remains disabled before full setup reaches `ready`.
- ☐ Enter invalid or normal pay-as-you-go key as Token Plan Key and confirm setup explains that CLI requires a Token Plan Key.
- ☐ Enter valid Token Plan Key.
- ☐ If CLI is missing, confirm install command preview uses the app-local prefix under Electron `userData`.
- ☐ Confirm there is no `Continue API-only` action.
- ☐ Click **Install MiniMax CLI** if prompted.
- ☐ Confirm setup reaches `MiniMax API and CLI are ready`.
- ☐ Confirm chat input becomes enabled only after setup reaches `ready`.
- ☐ Run `mmx auth status --output json`, `mmx quota`, and `mmx text chat --message "Reply with exactly: ok"` manually.
- ☐ Use **Reset Token Plan key** and confirm setup returns to Token Plan entry.
- ☐ Use **Reset all** and confirm setup returns to General API key entry.

Phase 3: CLI Agent Bridge

Goal: Build the structured bridge between Bubbles and the CLI worker brain using real process execution and normalized event handling.

Status: Completed for the current app. Phase 3 now uses the app-local MiniMax CLI as the real execution path, shares one Electron-main lifecycle for chat/avatar/task state, gates chat behind completed MiniMax setup, and keeps CLI failures visible without crashing the UI.

Main files:

- `packages/core/src/cli/cliBridge.ts`.
- `packages/core/src/cli/cliEventParser.ts`.
- `packages/core/src/cli/taskPacketBuilder.ts`.

- `packages/core/src/orchestration/orchestrator.ts`.
- `apps/desktop/src/main/ipc/taskIpc.ts`.
- `apps/desktop/src/renderer/components/TaskDrawer.tsx`.
- `apps/desktop/src/main/main.ts`.
- `apps/desktop/src/renderer/App.tsx`.

Implementation tasks:

- ☒ Implement `TaskPacket` builder that includes user text, active agent, `skills.md`, memory context, allowed tools, and approval policy.
- ☒ Implement real CLI process runner using Node `child_process.spawn`.
- ☒ Route panel chat through the shared `app:send-message` path so messages, active task, avatar state, and task drawer events stay synchronized.
- ☒ Execute MiniMax tasks through the app-local `mmx` binary under Electron `userData`; do not fall back to a global `mmx` for Phase 3 runtime tasks.
- ☒ Preflight MiniMax setup before CLI work and avoid unnecessary re-auth/retry when setup is already `ready`.
- ☒ Stream stdout/stderr as `task.partial_output` and `task.status` events.
- ☒ Normalize known CLI outputs into `CliEvent`, including plain stdout, structured task JSON, MiniMax pretty JSON responses, and MiniMax JSON errors.
- ☒ Support cancellation through process termination and `task.cancelled`.
- ☒ Add timeout handling for stalled CLI commands.
- ☒ Persist raw task log with secrets redacted.
- ☒ Show live task events in the task drawer.
- ☒ Mark MiniMax runtime auth/network/quota failures as setup health errors without deleting stored keys.
- ☒ Map CLI lifecycle events to avatar states:
 - task received: `thinking`.
 - task running: `working`.
 - approval required: `waiting_approval`.
 - missing info: `confused`.
 - error: `concerned`.
 - success: `celebrating`.
- ☒ Keep test fixture commands for automated tests only.

Acceptance criteria:

- ☒ Bubbles sends a structured task packet to a real CLI command.
- ☒ Task drawer streams status, partial output, errors, and result.
- ☒ User can cancel a running task.
- ☒ CLI errors are visible and do not crash the app.
- ☒ Avatar state follows task lifecycle.
- ☒ Chat stays disabled until MiniMax setup reaches `ready`.

Tests:

- ☒ Unit test task packet creation.
- ☒ Unit test event parser with stdout, stderr, JSON lines, malformed output, pretty JSON errors, and MiniMax JSON output.
- ☒ Integration test process runner with a local fixture command.
- ☒ Integration test cancellation.
- ☒ Integration test stalled command timeout and redacted task logs.
- ☒ Renderer test chat submission through the shared app message path.
- ☒ Renderer test Task Drawer status/error/result display.
- ☒ Manual UI smoke test from `/Users/dev/Documents/GitHub/bubbles-MVP` confirmed Phase 3 setup gating and panel/task surfaces.

Manual Phase 3 checklist:

- ☒ Launch from repo root with `corepack pnpm dev`.
- ☒ Open the Bubbles panel from the floating avatar.
- ☒ Confirm chat is disabled while MiniMax setup is incomplete.
- ☒ Confirm Settings shows the active MiniMax setup step.
- ☒ Confirm Task Drawer is visible and stable before any task.
- ☒ With setup ready, send a simple CLI task such as `Reply with exactly: ok`.
- ☒ Confirm the user message appears immediately.
- ☒ Confirm avatar state follows task lifecycle.
- ☒ Confirm Task Drawer shows task lifecycle events.
- ☒ Confirm final CLI result appears in chat without raw thinking JSON.
- ☒ Confirm CLI errors appear in chat/task drawer and do not crash the app.
- ☒ Confirm cancellation emits `task.cancelled`.

Phase 4: Agent Identity + Agent Birth

Goal: Create the default agents and support generating a new agent with `agent.json` and

`skills.md`, while keeping one universal Bubbles body.

Main files:

- Create `packages/core/src/agents/agentRegistry.ts`.
- Create `packages/core/src/agents/agentBirthService.ts`.
- Create `agents/general-assistant/agent.json` and `skills.md`.
- Create `agents/research-agent/agent.json` and `skills.md`.
- Create `agents/coding-agent/agent.json` and `skills.md`.
- Create `agents/email-calendar-assistant/agent.json` and `skills.md`.
- Create `agents/creative-minimax-helper/agent.json` and `skills.md`.
- Create `apps/desktop/src/renderer/components/AgentSwitcher.tsx`.
- Create `apps/desktop/src/renderer/screens/AgentBirthPreview.tsx`.

Implementation tasks:

- ☐ Define `agent.json` schema without visual style fields.
- ☐ Include fields for `id`, `name`, `role`, `badgeName`, `voiceStyle`, `allowedTools`, `memoryRules`, `safetyRules`, `responseStyle`, `skillsPath`, `timestamps`.
- ☐ Write default `skills.md` files for the five default agents.
- ☐ Implement registry `list/load/activate/create/update/archive`.
- ☐ Add agent switcher UI.
- ☐ Active agent updates `badge/name`, `voice style`, `response style`, `allowed tools`, and `prompt context` only.
- ☐ Implement Agent Birth request using MiniMax direct API.
- ☐ Generate proposed `agent.json` and `skills.md`.
- ☐ Show preview before creating files.
- ☐ Require approval before writing agent files.
- ☐ Store agent birth as a timeline event.

Acceptance criteria:

- ☐ Default agents appear in the switcher.
- ☐ Switching agents changes `badge/name` and `behavior instructions`.
- ☐ Switching agents does not change `avatar art` or `chat panel theme`.
- ☐ User can create a new agent after approving preview.
- ☐ New agent has valid `agent.json` and `skills.md`.

Tests:

- ☐ Unit test agent schema validation.
- ☐ Unit test agent registry load and activate.
- ☐ Unit test generated agent rejects visual customization fields.
- ☐ Integration test agent birth preview-to-create flow with mocked MiniMax response.
- ☐ Manual test create "coding agent for this project."

Phase 5: Memory Core + Timeline

Goal: Add local memory and timeline so Bubbles can remember user preferences, tasks, decisions, approvals, agents, connectors, and project context.

Main files:

- Create `packages/core/src/memory/memoryStore.ts`.
- Create `packages/core/src/memory/memoryExtractor.ts`.
- Create `packages/core/src/timeline/timelineStore.ts`.
- Create `packages/core/src/memory/schema.sql`.
- Create `apps/desktop/src/renderer/screens/MemoryTimeline.tsx`.
- Create `apps/desktop/src/main/ipc/memoryIpc.ts`.

Implementation tasks:

- ☐ Add SQLite database under app user data directory, not repo root.
- ☐ Create tables for memories, timeline events, task runs, approvals, connectors, and settings.
- ☐ Implement memory CRUD.
- ☐ Implement timeline append/list.
- ☐ Implement memory retrieval by type, agent, tags, and recent relevance.
- ☐ Implement memory extraction from user messages using MiniMax direct API.
- ☐ Add explicit user command handling for "remember X."
- ☐ Include relevant memories in `TaskPacket`.
- ☐ Store task summaries after CLI completion.
- ☐ Store approval decisions.
- ☐ Add UI for memory timeline and clear-memory action.

Acceptance criteria:

- ☐ User can say "Remember I like short plans."
- ☐ Bubbles stores the preference.
- ☐ Later tasks include the preference in memory context.

- ☐ Timeline shows agent creation, task runs, approvals, and memories.
- ☐ User can clear memory from settings.

Tests:

- ☐ Unit test SQLite migrations.
- ☐ Unit test memory CRUD.
- ☐ Unit test timeline append/list.
- ☐ Unit test memory selection for task packet.
- ☐ Integration test "remember preference" and later recall.

Phase 6: Real MCP/Connector Registry

Goal: Connect real tools through a hybrid connector registry: web research, local files/project folders, email, and calendar.

Main files:

- Create `packages/core/src/connectors/connectorRegistry.ts`.
- Create `packages/core/src/connectors/mcpClient.ts`.
- Create `packages/core/src/connectors/webSearchConnector.ts`.
- Create `packages/core/src/connectors/localFilesConnector.ts`.
- Create `packages/core/src/connectors/emailConnector.ts`.
- Create `packages/core/src/connectors/calendarConnector.ts`.
- Create `apps/desktop/src/renderer/screens/ConnectorSettings.tsx`.

Implementation tasks:

- ☐ Define connector model: id, name, type, enabled, authStatus, healthStatus, allowedAgents, requiredApproval, launchConfig.
- ☐ Implement MCP client wrapper for launching and talking to configured MCP servers.
- ☐ Implement web search connector using Tavily MCP if configured, with MiniMax search as fallback.
- ☐ Implement local files connector with explicit folder permission and read-only default.
- ☐ Implement email connector against a real Gmail/Outlook MCP path when credentials are configured.
- ☐ Implement calendar connector against a real Google/Outlook Calendar MCP path when credentials are configured.
- ☐ Add connector settings UI for connect, disconnect, health check, and permissions.
- ☐ Do not silently access email/calendar. First use must require connector setup.
- ☐ Keep fixture connectors for automated tests only, clearly marked as fixture mode.

Acceptance criteria:

- ☐ Connector settings shows web, local files, email, and calendar.
- ☐ Web research can call a real search provider when configured.
- ☐ Local files connector can read approved folders.
- ☐ Email connector can fetch latest email when configured.
- ☐ Calendar connector can read tomorrow's schedule when configured.
- ☐ Missing connector states are graceful and actionable.

Tests:

- ☐ Unit test connector registry.
- ☐ Unit test connector health states.
- ☐ Integration test MCP client with fixture server.
- ☐ Manual test Tavily or MiniMax web search.
- ☐ Manual test email/calendar if credentials are available.

Phase 7: Permission + Safety Layer

Goal: Ensure sensitive actions never happen silently and are always previewed in Bubbles.

Main files:

- Create `packages/core/src/approvals/approvalService.ts`.
- Create `packages/core/src/approvals/riskClassifier.ts`.
- Create `packages/core/src/security/secretRedactor.ts`.
- Create `apps/desktop/src/renderer/components/ApprovalModal.tsx`.
- Create `apps/desktop/src/main/ipc/approvalIpc.ts`.

Implementation tasks:

- ☐ Classify action risk for sending email, calendar edit, file write, shell command, CLI install, agent file create, and external data send.
- ☐ Generate approval previews with title, explanation, risk, affected target, and action payload.
- ☐ Block action execution until approval is recorded.
- ☐ Support approve, deny, and cancel.
- ☐ Map pending approval to `waiting_approval` avatar state.
- ☐ Store approval history in SQLite and timeline.
- ☐ Redact secrets in previews, logs, task drawer, memory, and timeline.
- ☐ For email reply, preview recipient, subject, and body.

- ☐ For calendar update, preview old and new event details.
- ☐ For file write, preview path and diff.
- ☐ For shell command or CLI install, preview command and reason.

Acceptance criteria:

- ☐ Bubbles never sends email without approval.
- ☐ Bubbles never edits calendar without approval.
- ☐ Bubbles never writes files without approval.
- ☐ Bubbles never installs CLI without approval.
- ☐ Approval denial stops the action and returns user-friendly status.
- ☐ Approval history appears in timeline.

Tests:

- ☐ Unit test risk classification.
- ☐ Unit test secret redaction.
- ☐ Unit test approval state transitions.
- ☐ Integration test email reply denied.
- ☐ Integration test calendar update approved.
- ☐ Integration test file write blocked until approval.

Phase 8: Real Capability Flows

Goal: Wire the main demo capabilities end to end using real integrations where configured: planning, research, coding, email, calendar, agent creation, and creative MiniMax.

Main files:

- Create `packages/core/src/orchestration/intentClassifier.ts`.
- Create `packages/core/src/orchestration/flowRouter.ts`.
- Create `packages/core/src/orchestration/responsePresenter.ts`.
- Create `packages/core/src/minimax/ttsService.ts`.
- Create `packages/core/src/minimax/creativeService.ts`.
- Update `apps/desktop/src/renderer/App.tsx`.

Implementation tasks:

- ☐ Implement intent classification for the supported task types.
- ☐ General task flow: route "plan my project" to general assistant and CLI.
- ☐ Research flow: route to research agent, web connector, source-aware response.

- ☐ Coding flow: route to coding agent, project context, CLI bridge, approval previews.
- ☐ Email read flow: route to email connector and summarize latest email.
- ☐ Email reply flow: draft only, then approval, then send.
- ☐ Calendar read flow: summarize schedule.
- ☐ Calendar update flow: draft update only, then approval, then apply.
- ☐ Agent birth flow: generate and preview new agent files.
- ☐ Creative MiniMax flow: run speech/image/vision tasks through MiniMax API or CLI.
- ☐ Response presenter rewrites technical output into nontechnical spoken summaries.
- ☐ Add MiniMax TTS for voice output, with a mute toggle.

Acceptance criteria:

- ☐ "Help me plan my Bubbles MVP" returns a clear spoken plan.
- ☐ "Research the best way to connect MCP tools" returns summary, sources, and uncertainty.
- ☐ "Create a coding agent for this project" previews and creates agent files after approval.
- ☐ "Read my last email" summarizes the latest email when connector is configured.
- ☐ "Reply that I can join" drafts and sends only after approval.
- ☐ "Check my calendar tomorrow" summarizes schedule.
- ☐ "Generate a friendly voice intro" produces or previews MiniMax audio.
- ☐ "Remember I like short plans" affects later responses.

Tests:

- ☐ Unit test intent classification examples.
- ☐ Integration test each flow with connector fixtures.
- ☐ Manual test real MiniMax planning/research.
- ☐ Manual test real email/calendar if credentials are available.
- ☐ Manual test creative MiniMax TTS.

Phase 9: Buildathon Polish, Packaging, And Demo Readiness

Goal: Stabilize, package, and polish the MVP into a credible buildathon product demo.

Main files:

- Create docs/demo_script.md.
- Create docs/setup_guide.md.
- Create apps/desktop/electron-builder.yml or equivalent packaging config.
- Create apps/desktop/src/renderer/screens/DemoModePanel.tsx.

Implementation tasks:

- ☐ Add demo script matching `docs/full_plan_v2.md`.
- ☐ Add setup guide for MiniMax General API key, MiniMax Token Plan Key, CLI, MCP connectors, email/calendar credentials, and troubleshooting.
- ☐ Add app settings for MiniMax status, CLI status, connector status, memory clear, voice toggle, and logs export.
- ☐ Add graceful degraded states:
 - MiniMax API unavailable.
 - CLI unavailable.
 - MCP connector unavailable.
 - Email/calendar auth missing.
 - Voice generation unavailable.
- ☐ Add visible "real integration" status indicators so judges can see what is live.
- ☐ Add crash-safe logging with secret redaction.
- ☐ Add packaging for macOS dev distribution.
- ☐ Run full manual demo path:
 - Open Bubbles as floating avatar.
 - Drag Bubbles to a new desktop position.
 - Click Bubbles to open the assistant panel.
 - Setup MiniMax.
 - Plan project.
 - Research MCP.
 - Create coding agent.
 - Read email.
 - Reply with approval.
 - Check calendar.
 - Save memory.
 - Ask follow-up that uses memory.
- ☐ Record known limitations in `docs/setup_guide.md`.

Acceptance criteria:

- ☐ MVP can be launched from a packaged macOS build or stable dev command.
- ☐ Demo path works end to end.
- ☐ Real integration statuses are visible.

- ☐ Missing integrations produce helpful instructions rather than broken UI.
- ☐ No secrets appear in logs, memory, task drawer, or timeline.
- ☐ The one universal sprite sheet remains the only avatar art system.

Tests:

- ☐ Run `pnpm test`.
- ☐ Run `pnpm typecheck`.
- ☐ Run `pnpm build`.
- ☐ Run Playwright smoke test for floating avatar, drag behavior, click-open assistant panel, setup, chat, approval modal, memory timeline, and connector settings.
- ☐ Manual macOS packaging smoke test.

Final Verification Checklist

- ☐ Floating avatar shell opens on macOS.
- ☐ Bubbles can be dragged around the desktop.
- ☐ Clicking Bubbles opens the Codex/Claude-style assistant panel.
- ☐ Assistant panel includes conversation history, active chat, task drawer, memory, settings, connector status, and approval surfaces.
- ☐ One Bubbles sprite sheet drives all avatar states.
- ☐ No agent-specific avatar skins, palettes, props, costumes, or chat themes exist.
- ☐ MiniMax dual-key setup works with separate secure Keychain storage.
- ☐ MiniMax CLI detection, Token Plan authentication, quota check, and verification work.
- ☒ CLI bridge streams real task events.
- ☐ Default agents load from local `agents/` folders.
- ☐ New agent creation previews `agent.json` and `skills.md`.
- ☐ Memory stores and recalls user preferences.
- ☐ Timeline shows tasks, approvals, memories, and agent birth events.
- ☐ Web research works through real configured provider.
- ☐ Email read/reply works through real configured connector or clearly reports missing auth.
- ☐ Calendar read/update works through real configured connector or clearly reports missing auth.
- ☐ All sensitive actions require approval.
- ☐ Voice + chat interaction works.
- ☐ Buildathon demo script can be completed.

Execution Order

Implement phases in order. Do not begin Phase 6 real connectors before Phase 7 approval architecture is at least scaffolded, because email/calendar/file actions need permission gates. During implementation, it is acceptable to build the approval service skeleton in Phase 6 and complete the UI in Phase 7.

Recommended commit sequence:

1. chore: scaffold desktop workspace
2. feat: add bubbles avatar shell
3. feat: add required minimax dual-key setup flow
4. feat: add cli bridge
5. feat: add agent registry
6. feat: add memory timeline
7. feat: add connector registry
8. feat: add permission previews
9. feat: wire mvp capability flows
- 10.chore: polish buildathon demo